

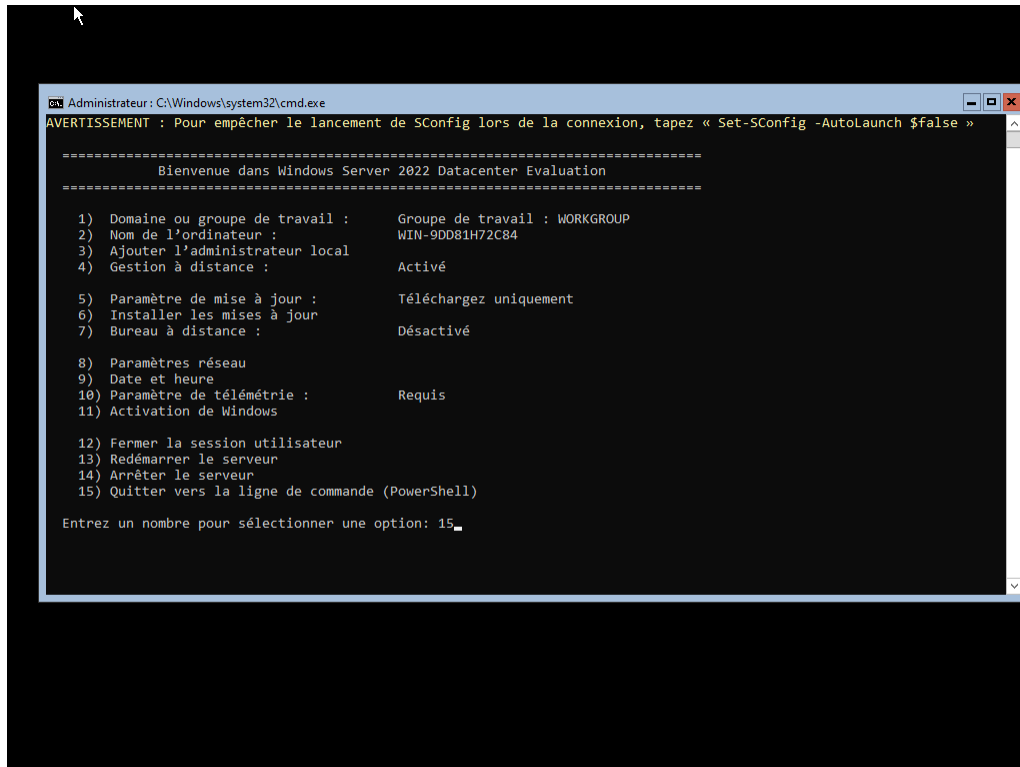
Automatiser la gestion d'active directory

Benjamin LEPRETRE

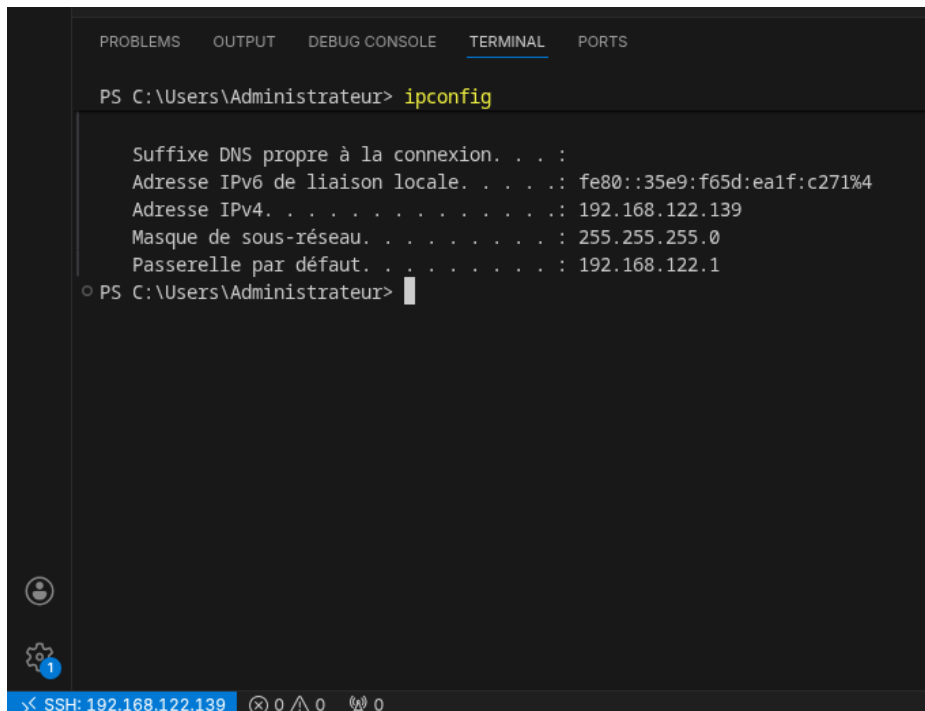
Mastère Expert en Système d'informations option cybersécurité Année 2

Intégration LDAP – Samba / Linux

- 1) Cloner et démarrer une machine virtuelle Windows Server Standard Core et ouvrir une session PowerShell locale ou à distance via Visual Studio Code.



Ouverture d'une fenêtre PowerShell depuis notre machine Windows core via Visual Studio Code :



The image shows a terminal window with a dark background. At the top, there are tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected), and 'PORTS'. The terminal prompt is 'PS C:\Users\Administrateur>'. The command 'ipconfig' has been entered and executed. The output shows network configuration details: 'Suffixe DNS propre à la connexion. . . . :', 'Adresse IPv6 de liaison locale. . . . : fe80::35e9:f65d:ea1f:c271%4', 'Adresse IPv4. : 192.168.122.139', 'Masque de sous-réseau. : 255.255.255.0', and 'Passerelle par défaut. : 192.168.122.1'. Below the output, the prompt 'PS C:\Users\Administrateur>' is visible with a cursor. At the bottom of the terminal, there is a status bar showing 'SSH: 192.168.122.139' and some connection icons.

```
PS C:\Users\Administrateur> ipconfig

Suffixe DNS propre à la connexion. . . . :
Adresse IPv6 de liaison locale. . . . : fe80::35e9:f65d:ea1f:c271%4
Adresse IPv4. . . . . : 192.168.122.139
Masque de sous-réseau. . . . . : 255.255.255.0
Passerelle par défaut. . . . . : 192.168.122.1
PS C:\Users\Administrateur>
```

2) Lister à l'aide de la commande Get- WindowsFeature les services, rôles et fonctionnalités installés sur le serveur :

Le résultat de ta commande Get-WindowsFeature montre que seuls les composants vitaux sont présents :

- **Services de fichiers et de stockage** (indispensable pour gérer les disques).
- **.NET Framework 4.8** (nécessaire pour exécuter les applications et scripts modernes).
- **Antivirus Microsoft Defender** (la sécurité de base même sans interface).
- **Windows PowerShell** (le seul et unique outil d'administration sur cette machine).

3) Identifier les services installés par défaut sur un serveur Core :

L'objectif d'un serveur **Core** est de réduire la "surface d'attaque".

- Par défaut, on ne trouve **aucun outil d'administration graphique** (pas de Gestionnaire de serveur local, pas d'Explorateur).
- Les services installés se limitent à l'infrastructure de base : stockage, PowerShell, et sécurité (Defender).

4) Identifier les services disponibles nécessaires au rôle d'annuaire :

Pour transformer ce serveur en contrôleur de domaine, tu dois repérer dans la liste complète (les éléments non cochés) :

- **AD-Domain-Services** (Services de domaine Active Directory).
- **DNS** (Serveur DNS), qui est quasi systématiquement couplé à l'AD pour la résolution des noms du domaine.

5) Installer le rôle d'annuaire correspondant :

La commande suivante va permettre d'installer le rôle d'annuaire sur le serveur :

`Install-WindowsFeature -Name AD-Domain-Services -IncludeManagementTools`

```
PS C:\Users\Administrateur> Install-WindowsFeature -Name AD-Domain-Services -IncludeManagementTools

Success Restart Needed Exit Code      Feature Result
-----
True      No                Success      {Services AD DS, Gestion de stratégie de g...

PS C:\Users\Administrateur>
```

- 6) Créer une nouvelle forêt Active Directory avec pour FQDN votrenom.lan à l'aide de la commande `Install-ADDSTForest` :

```
PS C:\Users\Administrateur> Install-ADDSTForest

applet de commande Install-ADDSTForest à la position 1 du pipeline de la commande
Fournissez des valeurs pour les paramètres suivants :
DomainName: ben.lan
SafeModeAdministratorPassword: *****
Confirmer SafeModeAdministratorPassword: *****

Le serveur cible sera configuré en tant que contrôleur de domaine et redémarré à la fin de cette opération.
Voulez-vous continuer en procédant à cette opération ?
[0] Oui [T] Oui pour tout [N] Non [U] Non pour tout [S] Suspendre [?] Aide (la valeur par défaut est « 0 ») : 0
```

- 7) Ouvrir une session sur le contrôleur de domaine fraîchement installé dès l'apparition du message Appuyez sur Ctrl-Alt-Suppr pour déverrouiller.

- 8) Vérifier que les rôles de serveur DNS et AD ont bien été installés sur le serveur :

```
PS C:\Users\Administrateur> Get-Service adws, dns, netlogon
```

Status	Name	DisplayName
Running	adws	Services Web Active Directory
Running	dns	Serveur DNS
Running	Netlogon	netlogon

```
PS C:\Users\Administrateur> Get-Module

ModuleType Version      Name                               ExportedCommands
-----
Manifest 1.0.1.0 ActiveDirectory {Add-ADCentralAccessPolicyMember, Add-ADCompute...
Manifest 3.1.0.0 Microsoft.PowerShell.Management {Add-Computer, Add-Content, Checkpoint-Computer...
Manifest 3.1.0.0 Microsoft.PowerShell.Utility {Add-Member, Add-Type, Clear-Variable, Compare-...
Script 2.0.0 PSReadline {Get-PSReadLineKeyHandler, Get-PSReadLineOption...
```

9) Lister les modules PowerShell chargés avec la cmdlet Get-Module, commenter :

L'exécution de la commande Get-Module montre que le module **ActiveDirectory** est désormais chargé en mémoire sur la session PowerShell. On peut noter plusieurs points importants :

- **Installation automatique** : Ce module a été installé sur le serveur au moment de l'ajout du rôle "Contrôleur de domaine Active Directory".
- **Importation** : Bien qu'il soit installé, il n'apparaît dans cette liste que s'il a été importé. PowerShell utilise l'**importation implicite**, c'est-à-dire qu'il charge le module dès qu'une commande comme Get-ADUser est appelée.
- **Capacités** : On voit dans la colonne ExportedCommands qu'il met à disposition une vaste bibliothèque de commandes spécifiques (ex: Add-ADComputer, Add-ADGroupMember) indispensables pour l'administration de l'annuaire.
- **Autres modules** : Les autres modules présents (Microsoft.PowerShell.Management et Utility) sont les modules de base de Windows permettant la gestion des fichiers, des services et des fonctions utilitaires système

10) Lister à présent tous les utilisateurs de l'annuaire avec la cmdlet Get-ADUser :

```
PS C:\Users\Administrateur> Get-ADUser -Filter *
```

DistinguishedName : CN=Administrateur,CN=Users,DC=ben,DC=lan
Enabled : True
GivenName :
Name : Administrateur
ObjectClass : user
ObjectGUID : 6b23ebbc-a264-4daa-8573-f20e8cda1443
SamAccountName : Administrateur
SID : S-1-5-21-2130295841-1263621094-1321313177-500
Surname :
UserPrincipalName :

DistinguishedName : CN=Invité,CN=Users,DC=ben,DC=lan
Enabled : False
GivenName :
Name : Invité
ObjectClass : user
ObjectGUID : 654167ff-f50d-40c1-8833-ff0eec9c5ed1
SamAccountName : Invité
SID : S-1-5-21-2130295841-1263621094-1321313177-501
Surname :
UserPrincipalName :

DistinguishedName : CN=krbtgt,CN=Users,DC=ben,DC=lan
Enabled : False
GivenName :
Name : krbtgt
ObjectClass : user
ObjectGUID : 22ebab09-a421-4719-b095-c23d1933e9b0
SamAccountName : krbtgt
SID : S-1-5-21-2130295841-1263621094-1321313177-502
Surname :
UserPrincipalName :

11) Lister de nouveau les modules PowerShell chargés, commenter :

```
PS C:\Users\Administrateur> Get-Module
```

ModuleType	Version	Name	ExportedCommands
Manifest	1.0.1.0	ActiveDirectory	{Add-ADCentralAccessPolicyMember, Add-ADComputerServiceAccount, Add-ADDomainControllerPasswordReplicationPolicy, Add-ADFineGrainedPasswordPolicySubject...}
Manifest	3.1.0.0	Microsoft.PowerShell.Management	{Add-Computer, Add-Content, Checkpoint-Computer, Clear-Content...}
Manifest	3.1.0.0	Microsoft.PowerShell.Utility	{Add-Member, Add-Type, Clear-Variable, Compare-Object...}
Script	2.0.0	PSReadline	{Get-PSReadLineKeyHandler, Get-PSReadLineOption, Remove-PSReadLineKeyHandler, Set-PSReadLineKeyHandler...}

« On constate qu'après avoir exécuté la commande Get-ADUser (Question 10), le module **ActiveDirectory** apparaît désormais comme "chargé" dans la liste des modules.

Cela illustre la fonctionnalité d'**importation implicite** de PowerShell : le système détecte qu'une commande appelée appartient à un module installé mais non encore chargé, et il l'importe automatiquement en mémoire pour l'utilisateur.

Il n'est donc pas strictement nécessaire d'utiliser la commandelette Import-Module ActiveDirectory manuellement avant de travailler, bien que cela reste possible pour s'assurer de la disponibilité des outils. »

12) Compter le nombre de cmdlets apportées par le module ActiveDirectory à l'aide des commandelettes Get-Command et Measure-Object :

```
PS C:\Users\Administrateur> Get-Command -Module ActiveDirectory | Measure-Object

Count      : 147
Average    :
Sum        :
Maximum    :
Minimum    :
Property   :
```

13) Lister les lecteurs associés aux fournisseurs en appelant la cmdlet Get-PSDrive et Identifier le point d'entrée du fournisseur « Active Directory »

En appelant la cmdlet Get-PSDrive, on visualise l'ensemble des lecteurs logiques disponibles dans la session. On identifie un lecteur nommé **AD** associé au fournisseur **ActiveDirectory**. Le point d'entrée du fournisseur est donc le lecteur virtuel **AD**:

Ce système permet d'explorer les objets de l'annuaire de la même manière qu'une arborescence de fichiers et de dossiers.


```

• PS C:\Users\Administrateur> Get-PSDrive

Name          Used (GB)  Free (GB) Provider      Root
----          -
AD             8,77      30,57  ActiveDire... //RootDSE/
Alias          Alias
C              8,77      30,57  FileSystem    C:\
Cert           Certificate \
D              4,71      0,00   FileSystem    D:\
Env            Environment
Function       Function
HKCU           Registry   HKEY_CURRENT_USER
HKLM           Registry   HKEY_LOCAL_MACHINE
Variable       Variable
WSMan          WSMAN

```

14) Explorer les objets à la racine du lecteur en appelant des cmdlets génériques :

```

• PS C:\Users\Administrateur> cd AD:
• PS AD:\> Get-ChildItem

Name          ObjectClass      DistinguishedName
----          -
ben           domainDNS        DC=ben,DC=lan
Configuration  configuration     CN=Configuration,DC=ben,DC=lan
Schema        dMD              CN=Schema,CN=Configuration,DC=ben,DC=lan
DomainDnsZones domainDNS        DC=DomainDnsZones,DC=ben,DC=lan
ForestDnsZones domainDNS        DC=ForestDnsZones,DC=ben,DC=lan

```

Grâce au fournisseur Active Directory, nous utilisons la commandelette générique `cd AD:` pour nous positionner à la racine de l'annuaire. En exécutant ensuite `Get-ChildItem`, nous listons les objets de premier niveau qui correspondent aux différentes partitions de la forêt (Schema, Configuration, et le domaine local ben.lan). L'affichage utilise le **DistinguishedName (DN)** pour identifier chaque partition de façon unique.

15) Lister les objets du domaine par la cmdlet Get-ChildItem DistinguishedName :

```
PS AD:\> Get-ChildItem -Path "DC=ben,DC=lan"
```

Name	ObjectClass	DistinguishedName
Builtin	builtinDomain	CN=Builtin,DC=ben,DC=lan
Computers	container	CN=Computers,DC=ben,DC=lan
Domain Controllers	organizationalUnit	OU=Domain Controllers,DC=ben,DC=lan
ForeignSecurityPr...	container	CN=ForeignSecurityPrincipals,DC=ben,DC=lan
Infrastructure	infrastructureUpdate	CN=Infrastructure,DC=ben,DC=lan
Keys	container	CN=Keys,DC=ben,DC=lan
LostAndFound	lostAndFound	CN=LostAndFound,DC=ben,DC=lan
Managed Service A...	container	CN=Managed Service Accounts,DC=ben,DC=lan
NTDS Quotas	msDS-QuotaContainer	CN=NTDS Quotas,DC=ben,DC=lan
Program Data	container	CN=Program Data,DC=ben,DC=lan
System	container	CN=System,DC=ben,DC=lan
TPM Devices	msTPM-Information...	CN=TPM Devices,DC=ben,DC=lan
Users	container	CN=Users,DC=ben,DC=lan

16) Lister à présent les utilisateurs du domaine :

```
PS AD:\> Get-ADUser -Filter *
```

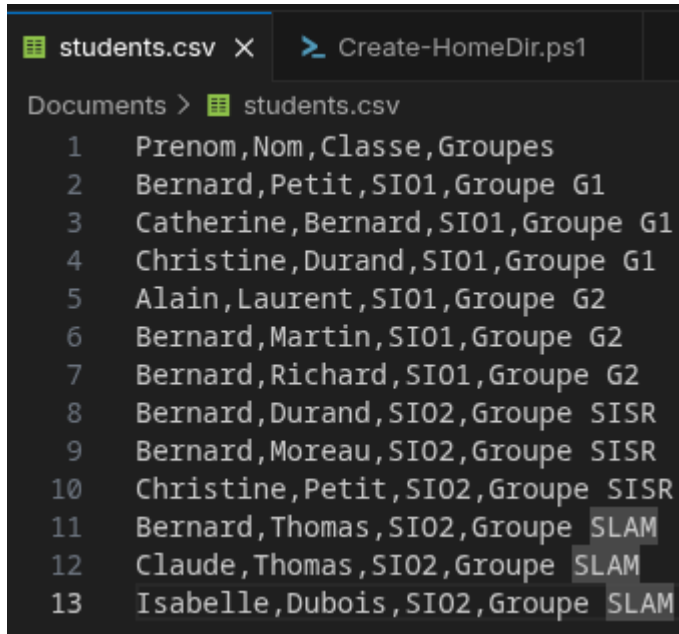
DistinguishedName : CN=Administrateur,CN=Users,DC=ben,DC=lan
Enabled : True
GivenName :
Name : Administrateur
ObjectClass : user
ObjectGUID : 6b23ebbc-a264-4daa-8573-f20e8cda1443
SamAccountName : Administrateur
SID : S-1-5-21-2130295841-1263621094-1321313177-500
Surname :
UserPrincipalName :

DistinguishedName : CN=Invité,CN=Users,DC=ben,DC=lan
Enabled : False
GivenName :
Name : Invité
ObjectClass : user
ObjectGUID : 654167ff-f50d-40c1-8833-ff0eec9c5ed1
SamAccountName : Invité
SID : S-1-5-21-2130295841-1263621094-1321313177-501
Surname :
UserPrincipalName :

DistinguishedName : CN=krbtgt,CN=Users,DC=ben,DC=lan
Enabled : False
GivenName :
Name : krbtgt
ObjectClass : user
ObjectGUID : 22ebab09-a421-4719-b095-c23d1933e9b0
SamAccountName : krbtgt
SID : S-1-5-21-2130295841-1263621094-1321313177-502
Surname :
UserPrincipalName :

1. Créer le script « Create-HomeDir.ps1 » qui lit le CSV et crée un dossier par classe, un sous-dossier par groupe, et un sous-sous-dossier par étudiant :

Fichier .csv :



```
students.csv X Create-HomeDir.ps1
Documents > students.csv
1 Prenom,Nom,Classe,Groupes
2 Bernard,Petit,SI01,Groupe G1
3 Catherine,Bernard,SI01,Groupe G1
4 Christine,Durand,SI01,Groupe G1
5 Alain,Laurent,SI01,Groupe G2
6 Bernard,Martin,SI01,Groupe G2
7 Bernard,Richard,SI01,Groupe G2
8 Bernard,Durand,SI02,Groupe SISR
9 Bernard,Moreau,SI02,Groupe SISR
10 Christine,Petit,SI02,Groupe SISR
11 Bernard,Thomas,SI02,Groupe SLAM
12 Claude,Thomas,SI02,Groupe SLAM
13 Isabelle,Dubois,SI02,Groupe SLAM
```

C'est un fichier CSV structuré avec les colonnes Prenom, Nom, Classe, et Groupes pour servir de base à l'automatisation.

2. Noter ici le contenu du script réalisé :

Script de création de dossier utilisateurs:

```
Documents > > Create-HomeDir.ps1
1  # On lit le fichier CSV
2  Import-Csv "$PSScriptRoot\students.csv" | ForEach-Object {
3
4      # On utilise "Prenom" sans accent
5      $Username = ($_.Prenom + '.' + $_.Nom).ToLower()
6
7      # Extraction du groupe (on récupère le dernier mot, ex: "G1")
8      $Groupe = $_.Groupes.Split(' ')[-1]
9
10     # Définition du chemin (C:\Classe\Groupe\Identifiant)
11     $Path = "C:\$(($_.Classe)\$Groupe\$Username"
12
13     # Vérification et création du dossier
14     if (-not (Test-Path -Path $Path)) {
15         New-Item -Path $Path -ItemType Directory -Force
16         Write-Host "Dossier créé : $Path" -ForegroundColor Green
17     } else {
18         Write-Host "Le dossier existe déjà : $Path" -ForegroundColor Yellow
19     }
20 }
```

3. Tester le script et vérifier le résultat obtenu à l'aide de la commande tree.

```
PS C:\Users\Administrateur\Documents> tree C:\SI01 /f ; tree C:\SI02 /f
Structure du dossier
Le numéro de série du volume est 00000201 CC34:39EE
C:\SI01
├── G1
│   ├── bernard.petit
│   ├── catherine.bernard
│   └── christine.durand
└── G2
    ├── alain.laurent
    ├── bernard.martin
    └── bernard.richard
Structure du dossier
Le numéro de série du volume est 000001C5 CC34:39EE
C:\SI02
├── SISR
│   ├── bernard.durand
│   ├── bernard.moreau
│   └── christine.petit
└── SLAM
    ├── bernard.thomas
    ├── claudethomas
    └── isabelle.dubois
```

4. Chercher et noter ci-dessous la commande permettant de créer un utilisateur :

Commande pour créer un nouvel utilisateur : New-ADUser

5. Chercher et Associer les paramètres et valeurs correspondants de la cmdlet :

Propriété	Paramètre / Option	Valeur(s) / Variable(s) utilisée(s)
Nom du compte utilisateur	-SamAccountName	\$Username (ex: bernard.petit)
Nom d'affichage de l'utilisateur	-DisplayName	"\$(\$_.Prenom) \$(\$_.Nom)"
Dossier personnel de l'utilisateur	-HomeDirectory	"\\SRV-AD-CORE\ \$Username"
Lecteur associé au dossier personnel	-HomeDrive	"H:"
Mot de passe par défaut de l'utilisateur	-AccountPassword	\$Password (Variable contenant 5securi+Y)
Changer le mot de passe à la connexion	-ChangePasswordAtLogon	\$true
Activer le compte	-Enabled	\$true

6. Compléter le script Create-Users.ps1 afin de créer les comptes utilisateurs :

Screen d'exécution du script :

```
PS C:\Users\Administrateur\Documents> .\Create-Users.ps1
Utilisateur créé avec succès : bernard.petit
Utilisateur créé avec succès : catherine.bernard
Utilisateur créé avec succès : christine.durand
Utilisateur créé avec succès : alain.laurent
Utilisateur créé avec succès : bernard.martin
Utilisateur créé avec succès : bernard.richard
Utilisateur créé avec succès : bernard.durand
Utilisateur créé avec succès : bernard.moreau
Utilisateur créé avec succès : christine.petit
Utilisateur créé avec succès : bernard.thomas
Utilisateur créé avec succès : claude.thomas
Utilisateur créé avec succès : isabelle.dubois
```

Script de création d'utilisateur :

```
Documents > .\Create-Users.ps1
1 # 1. Préparation du mot de passe sécurisé
2 $Password = "Securi+Y" | ConvertTo-SecureString -AsPlainText -Force
3
4 # 2. Lecture du fichier CSV
5 Import-Csv "$PSScriptRoot\students.csv" | ForEach-Object {
6
7     # Identifiant en minuscules (ex: bernard.petit)
8     $Username = ($_.Prenom + '.' + $_.Nom).ToLower()
9
10    # Définition de l'emplacement (OU)
11    $OU = "OU=($_.Classe),DC=ben,DC=lan"
12
13    # 3. Création de l'utilisateur dans l'AD
14    try {
15        New-ADUser -SamAccountName $Username `
16            -Name "$($_.Prenom) $($_.Nom)" `
17            -DisplayName "$($_.Prenom) $($_.Nom)" `
18            -AccountPassword $Password `
19            -HomeDirectory "\\SRV-AD-CORE\$Username" `
20            -HomeDrive "H:" `
21            -Path $OU `
22            -ChangePasswordAtLogon $true `
23            -Enabled $true
24
25        Write-Host "Utilisateur créé avec succès : $Username" -ForegroundColor Green
26    }
27    catch {
28        Write-Host "L'utilisateur $Username existe déjà ou erreur : $_.Exception.Message" -ForegroundColor Yellow
29    }
30 }
```


7. Commenter la gestion de la sécurité dans ce script

1. Sécurisation du mot de passe en mémoire

Dans le script, le mot de passe n'est pas transmis tel quel au système.

- On utilise la commande `ConvertTo-SecureString`.
- Cela transforme la chaîne de caractères "5ecuri+Y" en un objet chiffré que seul le système peut lire.
- **Note critique** : Même si le mot de passe est écrit en "clair" dans le code pour les besoins du TP, PowerShell exige qu'il soit converti avant d'être envoyé à l'Active Directory.

2. Le principe de Confidentialité (-ChangePasswordAtLogon)

C'est l'élément le plus important de ta gestion de la sécurité :

- L'option `-ChangePasswordAtLogon $true` force l'utilisateur à modifier son mot de passe dès sa toute première connexion.
- **Avantage** : L'administrateur (toi) connaît le mot de passe provisoire, mais une fois que l'étudiant s'est connecté, il est le **seul** à connaître son mot de passe définitif. Cela dégage ta responsabilité en cas d'action malveillante faite avec son compte.

3. Isolation des données personnelles

La configuration du **HomeDirectory** (`\\SRV-AD-CORE\Username`) prépare la sécurité des fichiers :

- En assignant un dossier unique basé sur le `$Username`, on permet la mise en place de permissions NTFS strictes.
- L'objectif est que seul l'étudiant concerné (et les administrateurs) puisse accéder au contenu de son lecteur **H:**.

8. Proposer une solution d'amélioration :

1. Automatisation de la création des Unités d'Organisation (OU)

Comme nous l'avons vu lors de ton erreur « Objet de l'annuaire non trouvé », le script échoue si l'arborescence AD n'est pas déjà prête.

- **L'amélioration** : Ajouter une fonction dans le script qui vérifie si l'OU (SIO1 ou SIO2) existe avec Get-ADOrganizationalUnit.
- **L'avantage** : Si l'OU est manquante, le script la crée tout seul avant d'ajouter l'utilisateur, ce qui rend le processus totalement autonome.

2. Gestion automatique des groupes de sécurité

Ton fichier CSV contient une colonne Groupes (ex: "Groupe G1", "Groupe SISR") que nous n'avons pas encore exploitée dans l'Active Directory.

- **L'amélioration** : Utiliser la cmdlet Add-ADGroupMember à l'intérieur de la boucle pour ajouter chaque étudiant à son groupe de classe ou de spécialité automatiquement.
- **L'avantage** : Cela permet de gérer les permissions réseau (accès aux dossiers, imprimantes) dès la création du compte, sans intervention manuelle supplémentaire.

3. Renforcement de la sécurité (Externalisation du mot de passe)

Actuellement, le mot de passe « 5ecuri+Y » est écrit en clair dans ton code source.

- **L'amélioration** : Ne pas stocker le mot de passe dans le script. On peut utiliser Read-Host -AsSecureString pour demander à l'administrateur de saisir le mot de passe provisoire au lancement du script.
- **L'avantage** : Le mot de passe ne reste pas gravé dans un fichier texte sur le disque dur, ce qui évite qu'un curieux puisse le lire.

4. Gestion des doublons (Idempotence)

Si tu relances le script, il affiche des messages d'erreur "Jaune" parce qu'il tente de recréer ce qui existe déjà.

- **L'amélioration** : Utiliser une condition if (-not (Get-ADUser -Filter "SamAccountName -eq '\$Username'")) avant de lancer la création.
- **L'avantage** : Le script devient "propre" : il ne traite que les nouveaux étudiants ajoutés au CSV sans essayer de modifier ceux qui sont déjà présents.

9. Concevoir un script Share-Homedir.ps1 qui va créer pour chaque utilisateur un partage réseau de son dossier personnel, nommé \$Username, sur lequel il aura tous les droits : (commandelette New-SmbShare) :

Succès du script :

```
PS C:\Users\Administrateur\Documents> .\Share-HomeDir.ps1

Partage créé avec succès pour : bernard.petit
Partage créé avec succès pour : catherine.bernard
Partage créé avec succès pour : christine.durand
Partage créé avec succès pour : alain.laurent
Partage créé avec succès pour : bernard.martin
Partage créé avec succès pour : bernard.richard
Partage créé avec succès pour : bernard.durand
Partage créé avec succès pour : bernard.moreau
Partage créé avec succès pour : christine.petit
Partage créé avec succès pour : bernard.thomas
Partage créé avec succès pour : claude.thomas
Partage créé avec succès pour : isabelle.dubois
```

Name	ScopeName	Path	Description
bernard.petit	*	C:\SI01\G1\bernard.petit	Dossier personnel de Bernard Petit
catherine.bernard	*	C:\SI01\G1\catherine.bernard	Dossier personnel de Catherine Bernard
christine.durand	*	C:\SI01\G1\christine.durand	Dossier personnel de Christine Durand
alain.laurent	*	C:\SI01\G2\alain.laurent	Dossier personnel de Alain Laurent
bernard.martin	*	C:\SI01\G2\bernard.martin	Dossier personnel de Bernard Martin
bernard.richard	*	C:\SI01\G2\bernard.richard	Dossier personnel de Bernard Richard
bernard.durand	*	C:\SI02\SI5R\bernard.durand	Dossier personnel de Bernard Durand
bernard.moreau	*	C:\SI02\SI5R\bernard.moreau	Dossier personnel de Bernard Moreau
christine.petit	*	C:\SI02\SI5R\christine.petit	Dossier personnel de Christine Petit
bernard.thomas	*	C:\SI02\SI5R\bernard.thomas	Dossier personnel de Bernard Thomas
claude.thomas	*	C:\SI02\SI5R\claude.thomas	Dossier personnel de Claude Thomas
isabelle.dubois	*	C:\SI02\SI5R\isabelle.dubois	Dossier personnel de Isabelle Dubois

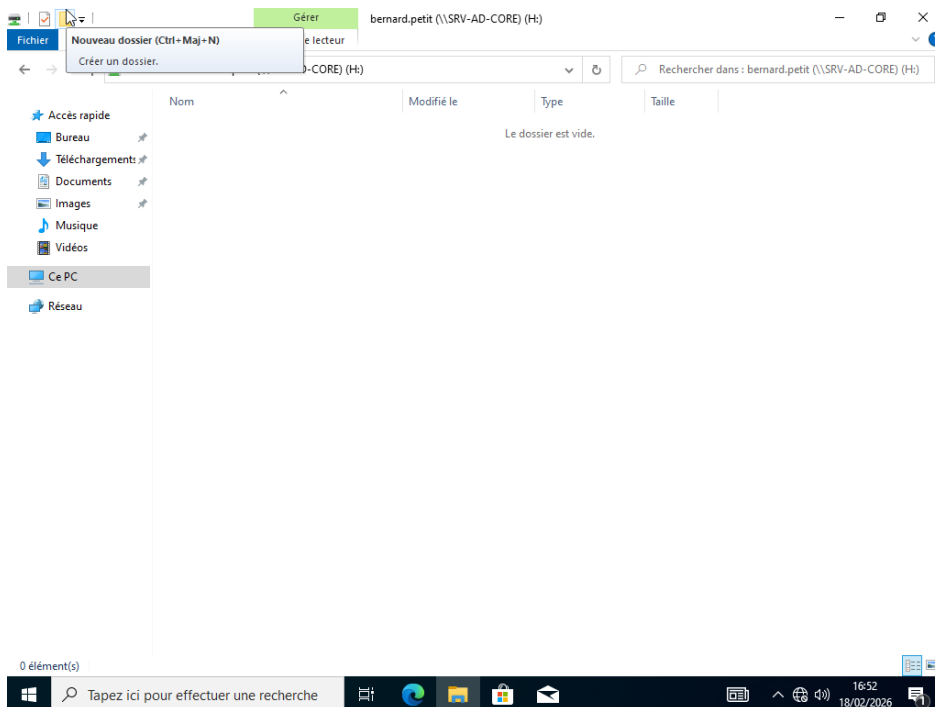
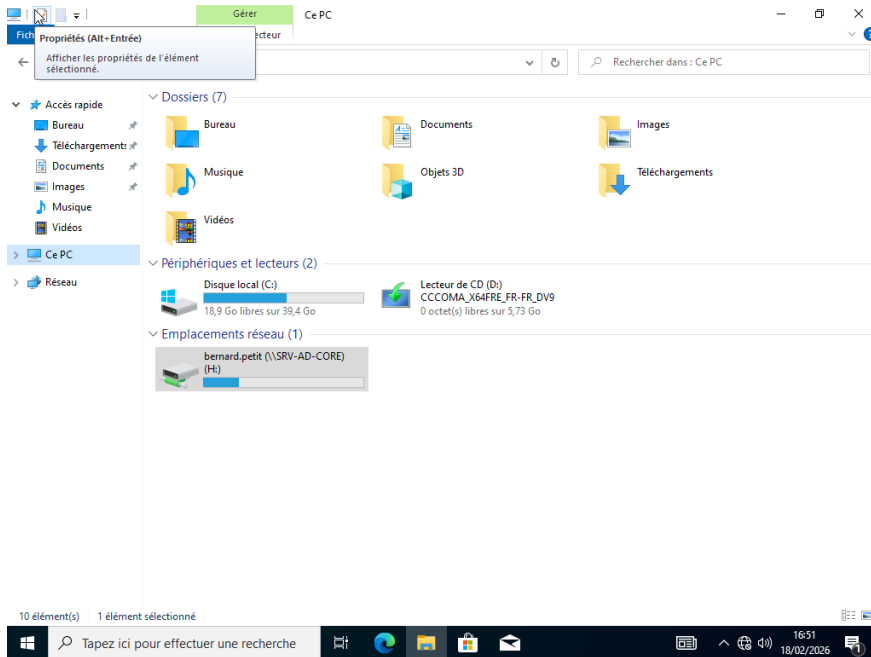
Script de partage du dossier perso de chaque utilisateur

:

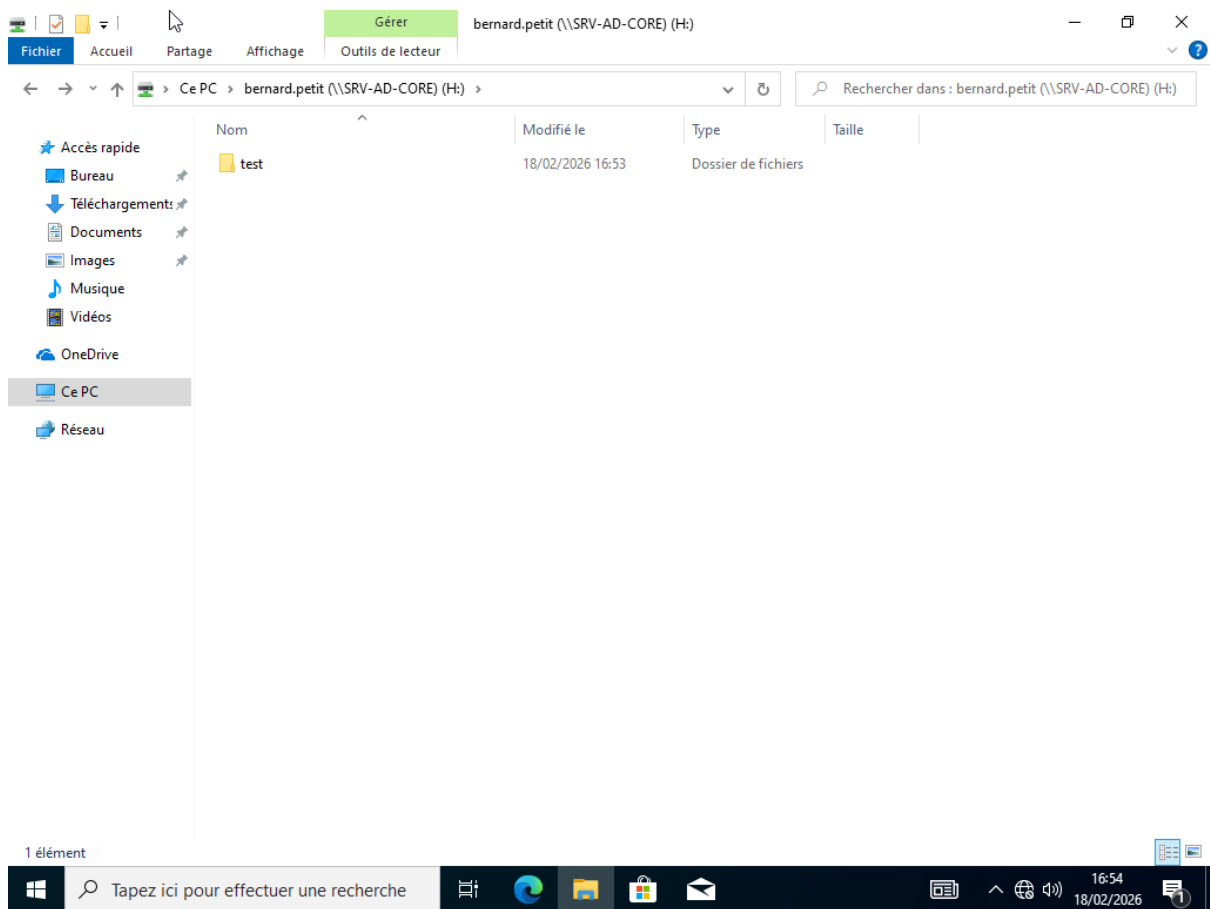
```
Documents > .\Share-HomeDir.ps1

1 # On lit le fichier CSV pour obtenir la liste des étudiants
2 Import-Csv "$PSScriptRoot\students.csv" | ForEach-Object {
3
4     # 1. Préparation des variables
5     $Username = ($_.Prenom + '.' + $_.Nom).ToLower()
6     $Groupe = $_.Groupes.Split(';')[0]
7
8     # Chemin local du dossier (celui créé à la Question 3)
9     $LocalPath = "C:\$($_.Classe)\$Groupe\$Username"
10
11     # 2. Création du partage SMB
12     try {
13         # New-SmbShare crée le partage réseau
14         # -Name : Le nom du partage (vu de l'extérieur)
15         # -Path : Le dossier physique sur le serveur
16         # -FullAccess : On donne tous les droits à l'utilisateur (Domaine\Username)
17         New-SmbShare -Name $Username `
18             -Path $LocalPath `
19             -FullAccess "ben\$Username" `
20             -Description "Dossier personnel de $(($_.Prenom) $(($_.Nom)))" `
21             -ErrorAction Stop
22
23         Write-Host "Partage créé avec succès pour : $Username" -ForegroundColor Green
24     }
25     catch {
26         Write-Host "Erreur ou partage déjà existant pour $Username : $(($_.Exception.Message))" -ForegroundColor Yellow
27     }
28 }
```

10. Tester l'ouverture d'une session et l'accès au lecteur H avec un compte ainsi créé.



11. Créer sous ce compte un fichier texte dans le lecteur H:, expliquer le résultat :



Il s'agit de notre dossier personnel donc nous avons les pleins droits dessus, on peut totalement y créer un dossier ou un fichier texte sans problème

- 12.** Depuis la session sysadmin, vérifier les autorisations existantes sur le dossier de l'utilisateur du compte avec la cmdlet Get-ACL : (ACL = liste de contrôle d'accès)

```
PS C:\Users\Administrateur> Get-Acl "C:\SI01\G1\bernard.petit" | Select-Object -ExpandProperty Access | Select-Object IdentityReference, FileSystemRights, AccessControlType
```

IdentityReference	FileSystemRights	AccessControlType
AUTORITE NT\Système	FullControl	Allow
BUILTIN\Administrateurs	FullControl	Allow
BUILTIN\Utilisateurs	ReadAndExecute, Synchronize	Allow
BUILTIN\Utilisateurs	AppendData	Allow
BUILTIN\Utilisateurs	CreateFiles	Allow
CREATEUR PROPRIETAIRE	268435456	Allow

- 13.** Compléter et exécuter le script Set-HomedirACL.ps1 suivant :

```
PS C:\Users\Administrateur\Documents> .\Set-HomedirACL.ps1
Droits configurÃ©s pour bernard.petit
Droits configurÃ©s pour catherine.bernard
Droits configurÃ©s pour christine.durand
Droits configurÃ©s pour alain.laurent
Droits configurÃ©s pour bernard.martin
Droits configurÃ©s pour bernard.richard
Droits configurÃ©s pour bernard.durand
Droits configurÃ©s pour bernard.moreau
Droits configurÃ©s pour christine.petit
Droits configurÃ©s pour bernard.thomas
Droits configurÃ©s pour claud
Droits configurÃ©s pour isabelle.dubois
```

```

Documents > Set-HomedirACL.ps1
1 # 1) On parcourt le CSV (vérifie si tu as gardé l'accent ou non sur "Prenom")
2 Import-Csv "$PSScriptRoot\students.csv" | ForEach-Object {
3
4     # Construction de l'identifiant
5     $Username = ($_.Prenom + '.' + $_.Nom).ToLower()
6     $Groupe = $_.Groupes.Split(' ')[-1]
7
8     # 2) Chemin du dossier (on utilise C:\ car c'est là que tu les as créés)
9     $Homedir = "C:\${$_Classe}\$Groupe\$Username"
10
11     # 3) Définition des droits : Utilisateur, Contrôle total, Autoriser
12     # On ajoute "ben\" pour préciser que c'est un compte du domaine
13     $Droits = "ben\$Username", 'FullControl', 'Allow'
14
15     # 4) Création de la règle d'accès (ACL)
16     $Regle = New-Object System.Security.AccessControl.FileSystemAccessRule -ArgumentList $Droits
17
18     # 5) Application de la règle
19     if (Test-Path $Homedir) {
20         $ACL = Get-Acl $Homedir           # On mémorise les droits existants
21         $ACL.SetAccessRule($Regle)       # On ajoute notre règle au paquet
22         Set-Acl -Path $Homedir -AclObject $ACL # On applique le tout sur le dossier
23
24         Write-Host "Droits configurés pour $Username" -ForegroundColor Green
25     }
26 }

```

14. Vérifier les nouvelles autorisations appliquées sur le dossier d'un utilisateur :

```

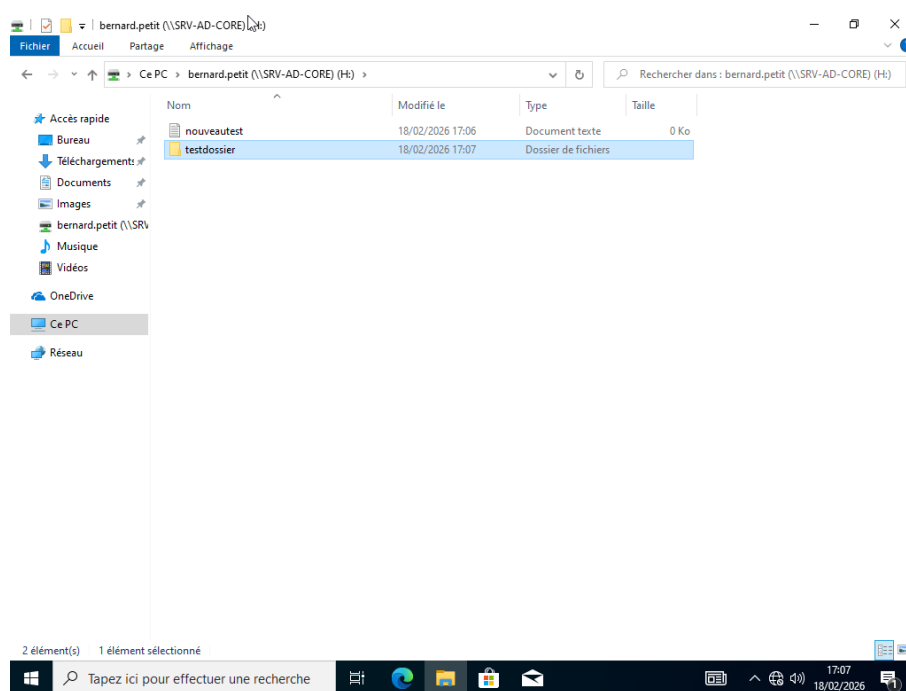
PS C:\Users\Administrateur> Get-Acl "C:\SI01\G1\bernard.petit" | Select-Object -ExpandProperty Access | Where-Object {$_.IdentityReference -like "*bernard.petit*"}

FileSystemRights : FullControl
AccessControlType : Allow
IdentityReference : BEN\bernard.petit
IsInherited       : False
InheritanceFlags  : None
PropagationFlags  : None

```

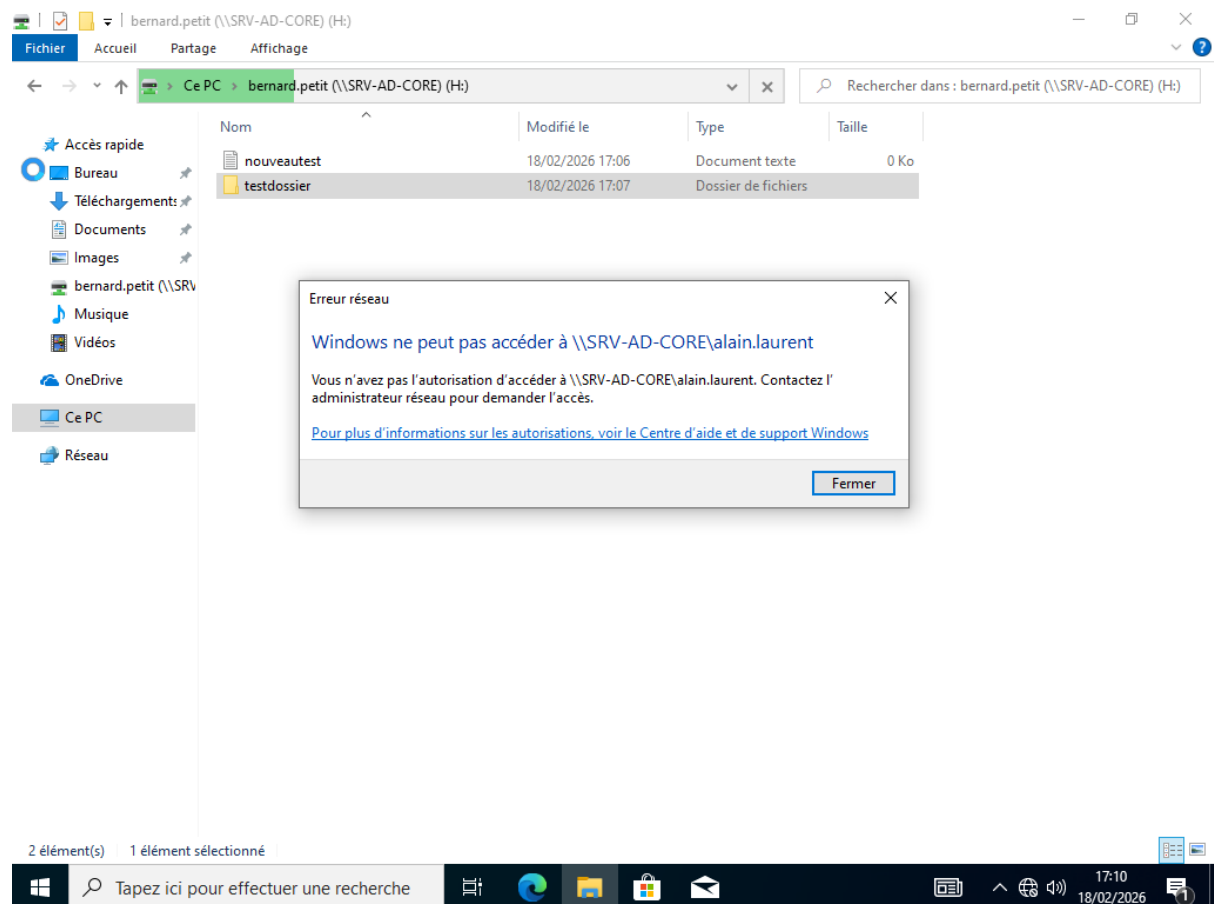
- **À la Q12** : L'utilisateur n'apparaissait pas ou n'avait que des droits hérités ("Read") via le groupe "Utilisateurs".
- **À la Q14** : L'utilisateur apparaît nommément avec le contrôle total sur son propre dossier, ce qui valide l'exécution du script Set-Acl.

15. Ouvrir une session avec cet utilisateur et tester à nouveau la création d'un document dans le lecteur H:



- **Succès de l'écriture** : La création du fichier réussit car l'utilisateur possède désormais les droits **FullControl** au niveau NTFS sur son dossier physique.
- **Vérification sur le serveur** : Si tu retournes sur ton serveur **SRV-AD-CORE** et que tu tapes la commande `ls C:\SIO1\G1\bernard.petit`, tu verras apparaître le fichier `Test_Bernard.txt` créé depuis le client.
- **Confidentialité** : Si tu essaies de faire la même chose en te connectant avec un autre étudiant (ex: alain.laurent), il ne pourra même pas entrer dans le dossier de Bernard, confirmant que tes ACL sont parfaitement étanches.

Erreur de connexion si tentative de connexion via un autre compte que celui du dossier partagé :



- **Preuve d'isolation** : « Ce message d'erreur prouve que le script Set-HomedirACL.ps1 a correctement rempli son rôle. »
- **Permissions NTFS** : « Bien que l'utilisateur soit authentifié sur le domaine, les listes de contrôle d'accès (ACL) configurées sur le serveur interdisent à tout utilisateur autre qu'Alain Laurent (et les administrateurs) de lire le contenu de ce dossier. »
- **Confidentialité** : « Cela garantit l'étanchéité totale des données personnelles entre les étudiants du groupe SIO1 et SIO2. »